# Product Requirements & Specification Document

## Project Name

**AgentX - Multi-Agent Collaboration Framework**

## Overview

AgentX is an open-source, research-focused framework enabling dynamic collaboration among specialized AI agents to solve complex legal domain tasks. Built with Python, FastAPI, Langchain, LangGraph, and OpenAI, AgentX orchestrates agentic behaviors, memory, and context-aware workflows.

---

## 1. Goals & Objectives

| Goal | Description |
|---|---|
| Multi-Agent Collaboration | Enable multiple specialized agents to interact and solve legal tasks |
| Dynamic Orchestration | Support context-aware, adaptive workflows |
| Agent Memory Integration | Equip agents with persistent, queryable memory |
| Open-Source & Extensible | Provide a modular, research-friendly codebase |
| API-Driven | Expose core functionalities via FastAPI endpoints |

---

## 2. Core Features

| Feature | Description |
|---|---|
| Agent Specialization | Agents with distinct legal expertise (e.g., contract review, case analysis) |
| Agent Memory | Long/short-term memory for context retention and recall |
| Workflow Orchestration | LangGraph-powered dynamic task routing and agent coordination |
| API Interface | FastAPI endpoints for task submission, status, and results |
| Extensibility | Plug-in architecture for new agents and workflows |
| Logging & Traceability | Track agent interactions and workflow steps |

---

## 3. User Stories

| As a… | I want to… | So that… |
|---|---|---|
| Legal Researcher | Submit complex legal queries to AgentX | I receive comprehensive, multi-perspective answers |

| | | |
|---|---|---|
| Developer | Add new agent types or workflows | I can extend AgentX for new legal domains |
| Stakeholder | Monitor agent collaboration and workflow progress | I can ensure transparency and reliability |

---

## 4. Technical Specifications

### 4.1 Architecture

```
[Client] <-> [FastAPI API] <-> [Orchestrator (LangGraph)] <-> [Agents (Langchain, C
                                                    |
                                              [Memory Store]
```

### 4.2 Technology Stack

| Component | Technology |
|---|---|
| API Layer | FastAPI |
| Orchestration | LangGraph |
| Agent Framework | Langchain |
| LLMs | OpenAI API |
| Memory | Vector DB / In-Memory (e.g., FAISS, Redis) |
| Language | Python |

### 4.3 Key Modules

| Module | Responsibilities |
|---|---|
| API | Task submission, status, results, agent management |
| Orchestrator | Workflow management, agent routing, context propagation |
| Agent | Task execution, memory access, inter-agent messaging |
| Memory | Store/retrieve agent context and knowledge |
| Logging | Record interactions, errors, and workflow traces |

---

## 5. Functional Requirements

| ID | Requirement |
|---|---|
| FR1 | System must accept legal tasks via REST API |
| FR2 | Orchestrator must dynamically assign tasks to appropriate agents |
| FR3 | Agents must access and update their memory during task execution |
| FR4 | System must support adding/removing agents without downtime |
| FR5 | API must provide endpoints for task status, results, and logs |

| | |
|---|---|
| FR6 | All agent interactions must be logged and traceable |
| FR7 | System must handle concurrent task submissions |

---

## 6. Non-Functional Requirements

| ID | Requirement |
|---|---|
| NFR1 | Response time for standard tasks < 5 seconds (excluding LLM latency) |
| NFR2 | System must be modular and extensible |
| NFR3 | Codebase must be open-source and well-documented |
| NFR4 | Ensure data privacy and secure API endpoints |
| NFR5 | Support for local and cloud deployment |

---

## 7. API Endpoints (Sample)

| Method | Endpoint | Description |
|---|---|---|
| POST | /tasks | Submit new legal task |
| GET | /tasks/{id} | Get task status/result |
| GET | /agents | List available agents |
| POST | /agents | Register new agent |
| GET | /logs/{task_id} | Retrieve task logs |

---

## 8. Example Workflow (Pseudocode)

```
# Submit task
POST /tasks { "query": "Analyze contract X for risks" }

# Orchestrator assigns to ContractReviewAgent
# Agent retrieves memory/context
# Agent may delegate to RiskAnalysisAgent
# Agents collaborate, update memory, return result

# Client polls /tasks/{id} for result
```

---

## 9. Milestones & Deliverables

| Milestone | Deliverable | Timeline |
|---|---|---|
| Core Framework | API, Orchestrator, Agent base, Memory | Week 2 |

| | | |
|---|---|---|
| Specialized Agents | Contract, Case, Risk Analysis agents | Week 4 |
| Dynamic Workflows | LangGraph integration, multi-agent flows | Week 6 |
| Logging & Extensibility | Logging, agent/plugin system | Week 7 |
| Documentation & Release | Open-source repo, usage docs | Week 8 |

---

## 10. Open Issues & Risks

| Issue/Risk | Mitigation |
|---|---|
| LLM API latency/cost | Caching, batching, local LLM support |
| Legal data privacy | Secure storage, API authentication |
| Workflow complexity | Modular design, extensive testing |

---

## 11. Success Criteria

- Multi-agent workflows solve complex legal tasks end-to-end
- Agents demonstrate memory and context awareness
- System is open-source, documented, and extensible
- API is stable and supports concurrent users

---

**End of Document**